Angular en una página

Cómo lo soporta TypeScript · Cómo funciona (MVC) · Cómo se conecta con un backend





1 · ¿Qué es Angular?



Soy un framework de Google para crear aplicaciones web de una sola página (SPA). Divido la pantalla en componentes reutilizables y traigo todo incluido: rutas, formularios y conexión HTTP.

Google	Framework	Componentes	Angular CLI
Lo creó y mantiene	Completo, todo incluido	Piezas reutilizables	Crea y compila con `ng`

2 · ¿Cómo lo soporta TypeScript?

Estoy escrito en TypeScript. El CLI lo compila a JavaScript para que el navegador lo entienda.

🏗️ **Clases y decoradores:** cada componente es una clase marcada con `@Component`.

🛡️ **Tipado:** los modelos son `interface`; los errores salen al escribir.

💉 **Inyección de dependencias:** los servicios se piden por tipo.

🔗 **Versiones:** cada Angular exige un rango de TypeScript.

```
@Component({ selector: 'app-lista',
  templateUrl: './lista.html' })
export class ListaComponent {
  titulo: string = 'Estudiantes';
}
```

3 · ¿Cómo funciona (MVC)?

📦 **Modelo**	Datos y lógica: `interface` y **servicios**.
🖼️ **Vista**	Lo que ve el usuario: plantilla `.html` y `.css`.
🎮 **Controlador**	Clase `.ts`: responde a eventos y prepara datos.

`{{ dato }}` Muestra un valor	`[propiedad]` Componente → HTML
`(evento)` Clic o teclado → componente	`[(ngModel)]` Ambos sentidos

💡 **Se parece a MVC, pero Angular se basa en componentes y enlace de datos (más cercano a MVVM).**

4 · ¿Cómo se conecta con un backend?

🖥️ **Angular**
Componente → Servicio → `HttpClient`

⇄ **HTTP + JSON**

⚙️ **API REST**
Node.js, Spring Boot…

⇄ **consultas**

🗄️ **Base de datos**
Guarda los datos

```
// servicio
@Injectable({ providedIn: 'root' })
export class EstudianteService {
  private http = inject(HttpClient);
  listar() { return this.http.get<Estudiante[]>('/api/estudiantes'); }
}  // en la plantilla: @for (e of estudiantes$ | async; track e.codigo) { {{ e.nombre }} }
```

📡 **Observable** La respuesta llega después; el pipe `async` la muestra.	🛣️ **Verbos** GET consulta, POST crea, PUT modifica, DELETE elimina.	🚧 **CORS** El backend debe permitir el origen de Angular.	🔐 **Interceptores** Añaden el token a cada petición.

En resumen

TypeScript 🛡️ da tipos y estructura

Angular 🖥️ muestra la interfaz

Backend ⚙️ guarda datos y responde en JSON